VINUNIVERSITY

Data Annotation 3D Object Detection: từ point cloud đến cuboid

Ngày 13 · Họ model, ground truth từ LiDAR, và quy trình pre-label

AI20K · AI in Action

VinUniversity ·

HÃY SUY NGHĨ...

“Một robotaxi không nhìn thấy phố. Nó chỉ tin những cuboid mà người gắn nhãn đã quyết định là thật.”

Giữ câu hỏi này trong đầu mỗi khi một box model trông gần đúng

Nội dung bài học

1. Các bài toán trên point cloud
2. Model nhìn point cloud thế nào?
3. Các thuật toán 3D detection điển hình
4. Camera có dự đoán được 3D không?
5. Nhắc lại Ngày 12
6. Quy trình pre-label bằng pretrained model

Đầu ra cuối buổi

Sau buổi lý thuyết, học viên có thể:

1. Gọi tên các họ 3D detector và đánh đổi giữa tốc độ với hình học
2. Tách hai việc: xe có thể chạy bằng camera; ground truth 3D của bài toán đó vẫn là LiDAR
3. Đọc output của detector thành 7 số cuboid đã học Ngày 12
4. Mô tả pipeline pre-label và nhận ra lỗi hệ toạ độ trên cả một batch

Buổi lab 4 giờ sau sẽ mở prediction và sửa cuboid trên CVAT. Hôm nay dừng ở cách model sinh ra box, và cách một người gắn nhãn đọc box đó.

Lưu ý: Rule nghiệp vụ vẫn là guideline VinFast. Số trong các paper chỉ để hiểu model, không phải kích thước để gắn nhãn.

Các bài toán trên point cloud

01

Detection là việc của hôm nay. Registration và SLAM là hai bài toán khác nhau.

Sáu việc người ta làm với point cloud

VINUNIVERSITY

Bài toán	Model trả về cái gì
3D classification	Một cụm điểm đã cắt sẵn, một class. Không định vị, không vẽ box.
3D detection	**Cả scene: tìm object, trả class và cuboid. Việc của hôm nay.**
3D segmentation	Một class cho từng điểm, không chỉ một hộp bao ngoài.
3D tracking	Cùng một object qua nhiều frame, giữ track ID. Để Ngày 14.
Registration	Một phép biến đổi căn hai cloud về cùng hệ toạ độ.
SLAM	Quỹ đạo của xe và bản đồ theo thời gian. Registration chỉ là một mô-đun.

Classification nhận một object đã cắt (benchmark kiểu ModelNet40). Detection phải tự tìm object trong cả giao lộ — annotator làm việc ở tầng này.

Ví dụ: 3D classification

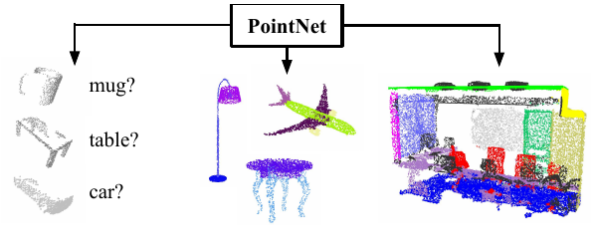


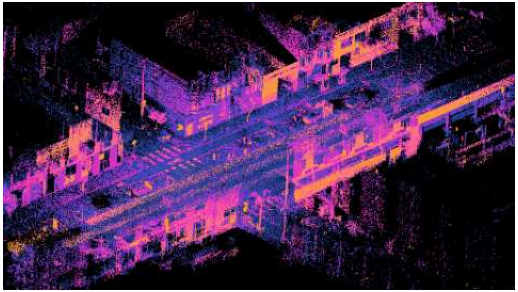
Figure 1, Qi et al., PointNet, CVPR 2017. Cột Classification: một cụm đã cắt, một class.

Một cụm điểm của đúng một vật. Đường, vỉa hè và xe khác đã bị bỏ ra ngoài.

Một class. Không có vị trí trên đường, không có cuboid. Annotator hiếm khi làm riêng việc này.

Dùng khi nào. Nhận dạng một vật đã cắt sẵn, như linh kiện hoặc đồ trong kho. Robotaxi ít dùng vì scene chưa được cắt.

Ví dụ: 3D detection



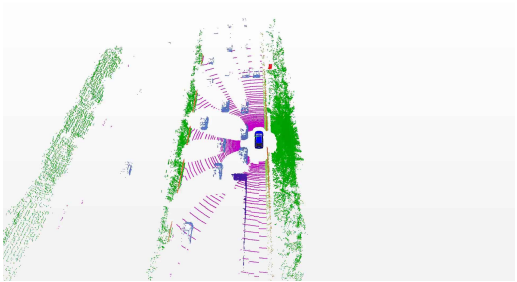
Giao lộ thật, Ouster OS1-64. Wikimedia Commons, CC BY 4.0.

> Cả scene: hàng trăm nghìn điểm, nhiều vật lẫn mặt đường.

> Mỗi vật một cuboid và một class. Model phải tự tìm vật trước khi gọi tên. Đây là việc của hôm nay, và là đầu ra người gắn nhãn sửa.

Dùng khi nào. Tìm xe, người, xe đạp trên cả giao lộ để tránh va chạm. Đây là việc người gắn nhãn sửa.

Ví dụ: 3D segmentation



SemanticKITTI, Behley et al. Ảnh trên semantic-kitti.org.

Mỗi màu là một class của điểm.

Cùng một scene như detection.

Một class cho từng điểm: xe, đường, người, nhà. Biên của vật sắc hơn một cuboid, và không có heading.

Dùng khi nào. Biết điểm nào là đường, vỉa hè hay xe, để xác định vùng chạy được. Không có hướng đầu.

Ví dụ: 3D tracking



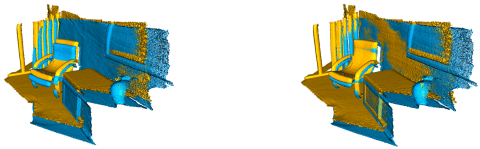
KITTI Tracking, Geiger et al., cvlibs.net. Mỗi màu là một ID xuyên các frame.

> Nhiều frame liên tiếp, mỗi frame đã có cuboid.

> Một ID gắn xuyên thời gian. Cuboid từng frame là detection. Giữ ID là tracking — để Ngày 14.

Dùng khi nào. Giữ cùng một xe qua nhiều frame để biết nó đang đi đâu. Để Ngày 14.

Ví dụ: registration



Trái: hai cloud lệch. Phải: sau khi căn. Không có nhãn vật.

Hai (hoặc vài) point cloud của cùng một chỗ, hệ toạ độ chưa trùng.

Một phép xoay và tịnh tiến. Chưa có bản đồ, chưa có quỹ đạo của xe.

Dùng khi nào. Ghép hai lần quét cho trùng hệ trục. Chưa đặt tên vật.

Ví dụ: SLAM



KITTI Odometry, Geiger et al., cvlibs.net. Xanh: quỹ đạo ước lượng. Xanh dương: GPS.

Một chuỗi scan lúc xe đang chạy.

Bản đồ của con đường và quỹ đạo của xe. Registration căn từng cặp scan là một bước bên trong, cùng với odometry và loop closure.

Dùng khi nào. Vừa định vị xe vừa dựng bản đồ khi xe đang chạy, kể cả nơi GPS yếu.

Registration và SLAM

- Bài toán hình học: căn hai (hoặc vài) point cloud
- Đầu ra là một phép xoay và tịnh tiến
- Thuật toán điển hình: ICP, NDT
- Dùng khi ghép hai lần quét cho khớp nhau

- Bài toán hệ thống: vừa định vị xe vừa dựng bản đồ
- Chạy liên tục theo thời gian, có quỹ đạo
- Bên trong có scan matching, odometry, loop closure, tối ưu pose graph
- Ngày 12 nhắc SLAM như ứng dụng indoor của LiDAR

Scan matching trong SLAM chính là registration. Gọi registration là SLAM thì mất phần bản đồ và quỹ đạo — hai đầu ra mà một phép căn hai cloud không có.

02

Model nhìn point cloud thế nào?

Điểm thưa, không có lưới. Mỗi cách biểu diễn kéo theo một họ kiến trúc.

Vì sao đám mây điểm khó hơn một tấm ảnh

VINUNIVERSITY

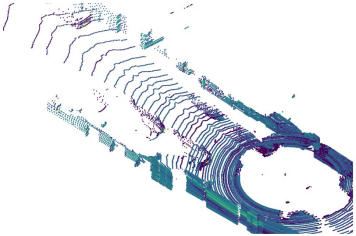
Lưu ý:

- Thưa và không đều: dày ở gần, thưa ở xa
- Không có lưới pixel cố định
- Đảo thứ tự điểm thì cảnh không đổi
- Một frame có hàng trăm nghìn điểm
- Vật bị che chỉ còn một mặt điểm

Làm sao biến một tập điểm lộn xộn thành đặc trưng mà một detection head đọc được?

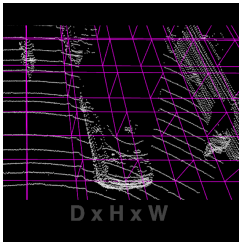
Câu trả lời chia theo **cách biểu diễn đầu vào**: giữ nguyên điểm, cắt thành voxel, nén thành cột, hay để attention tự hỏi.

Ba dạng detector trên xe dùng



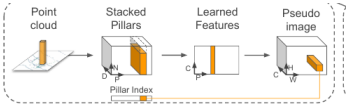
Điểm thô

Từng điểm (x, y, z). Chưa có ô.



Voxel

Cắt không gian thành khối 3D.



Pillar / BEV

Cột đứng, rồi thành ảnh từ trên.

Ảnh 3D trong paper. Điểm và pillar: Figure 2, Lang et al., PointPillars, CVPR 2019. Voxel: Figure 2, Zhou và Tuzel, VoxelNet, CVPR 2018. Ba slide sau phóng từng ảnh.

Điểm thô, nhìn 3D

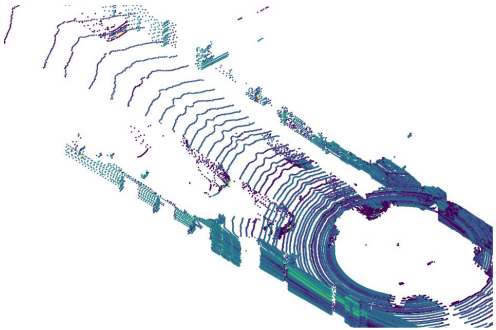


Figure 2, Lang et al., PointPillars, CVPR 2019. Point cloud KITTI nhìn 3D. Đường và xe là từng điểm, chưa có ô.

Một tập (x, y, z). Đảo thứ tự điểm thì vẫn là chiếc xe đó. Không có ô, không có pixel.

PointNet++ nhóm các điểm lân cận rồi mới nhận ra đây là một xe. Giữ hình học, chậm khi scene là cả con phố.

Voxel, nhìn 3D

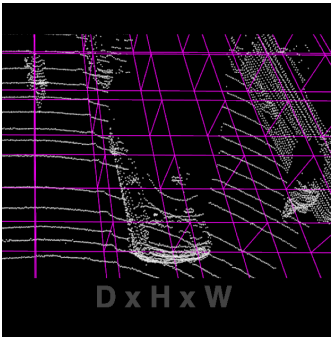


Figure 2, Zhou và Tuzel, VoxelNet, CVPR 2018. Lưới 3D phủ lên point cloud. Chỉ ô có điểm được giữ.

Không gian cắt thành các khối đều. Điểm của chiếc xe rơi vào một nắm ô. Phần lớn ô trên phố không có điểm.

SECOND chỉ tính ô có điểm. CenterPoint bản voxel cũng xuất phát từ lưới này, rồi tìm tâm xe trên BEV.

Pillar, nhìn 3D

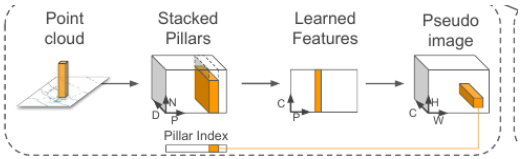


Figure 2, Lang et al., PointPillars, CVPR 2019. Cột cam đứng từ mặt đường lên. Nhiều cột xếp lại, rồi thành ảnh giả nhìn từ trên.

Chiều cao bị gộp sớm. Kết quả là một ảnh giả nhìn từ trên xuống. PointPillars đưa ảnh đó vào CNN 2D.

DSVT và fusion vẫn đọc voxel hoặc pillar. Chúng đổi cách tính, rồi vẫn ra cuboid của chính chiếc xe này.

Ba bước để có một box chạy trên xe

Ba dạng ở slide trước. Mỗi bước sửa một lỗi của bước trước.

1. **PointNet++ đọc điểm lân cận.** Model thấy hình khối của xe. Trên cả phố, hàng trăm nghìn điểm, cách này chậm.
2. **SECOND bỏ ô trống.** Xe vẫn nằm trong vài voxel có điểm. Phần đường không có vật không bị tính. Nhanh hơn, vẫn là lưới 3D.
3. **PointPillars nén mỗi cột thành một ô BEV.** CNN 2D, thứ ngành ảnh đã chạy rất nhanh, vẽ cuboid kịp trên xe. Chiều cao chi tiết bị gộp từ bước này.

Ba bước này là lý do annotator gặp PointPillars đầu tiên: nó là checkpoint pre-label phổ biến, và output đã là cuboid.

Sau khi đã có box, người ta sửa gì

Box đã hiện trên BEV. Bốn việc sau không tạo ra một loại nhãn mới.

1. **CenterPoint** tìm xe như một tâm trên BEV, rồi hồi quy kích thước, chiều cao và heading. Bỏ anchor.
2. **PV-RCNN++** quay lại điểm thật quanh box đó để sửa góc. Class có thể đã đúng trong khi cạnh vẫn lệch.
3. **VoxelNeXt và DSVT** tính trên chỗ còn điểm, hoặc bằng attention, để đỡ làm đặc cả bản đồ. Cuboid ra vẫn là bảy số.
4. **BEVFusion** thêm ảnh camera vào cùng mặt BEV, để phân biệt bus với xe tải. Hình học mét vẫn từ LiDAR.

Các thuật toán 3D detection điển hình

03

Ba mốc đáng nhớ

2019

PointPillars — pillar, realtime, checkpoint pre-label

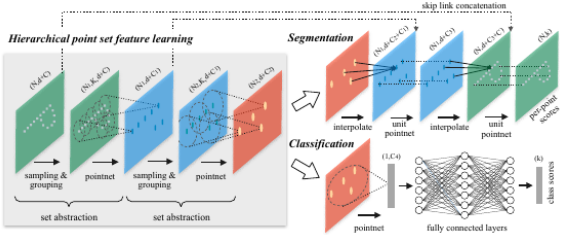
2021

CenterPoint — object là một tâm trên BEV

2023–26

Thưa, Transformer-based — cùng vẫn ra cuboid

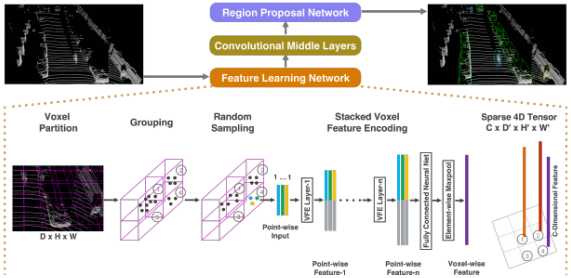
Point-based: PointNet, PointNet++, PointRCNN



- PointNet (2017): mỗi điểm qua cùng một MLP, rồi max-pooling. Đảo thứ tự điểm, kết quả không đổi.
- PointNet++ thêm lân cận: lấy mẫu rồi nhóm điểm quanh từng tâm.
- PointRCNN (2019) đề xuất box ngay từ điểm. Giữ hình học sắc, chậm trên scene đường phố.

Figure 2, Qi et al., PointNet++, NeurIPS 2017. PointNet: Qi et al., CVPR 2017. PointRCNN: Shi et al., CVPR 2019.

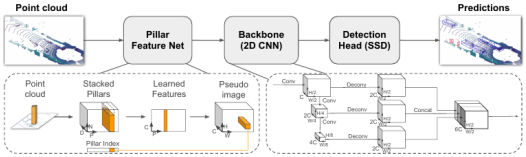
Voxel-based: VoxelNet rồi SECOND



- VoxelNet (2018) cắt không gian thành các ô đều, mã hoá điểm trong từng ô, rồi tích chập 3D.
- Phần lớn ô trống. Tích chập dày đặc trả giá cho khoảng không.
- SECOND chỉ tính ô có điểm — sparse convolution. Vẫn là lưới 3D, nhanh hơn nhiều.

Figure 2, Zhou và Tuzel, VoxelNet, CVPR 2018. SECOND: Yan et al., Sensors 2018.

PointPillars: bốn bước từ cột đến hộp



1. **Cột đứng.** Không cắt theo *z*. Mọi điểm trong một ô mặt đất rơi vào cùng một pillar.
2. **PointNet nhỏ.** Trong mỗi cột, một mạng học ra một vector. Đây là đặc trưng học được, không phải ảnh làm sẵn.
3. **Ảnh giả.** Rải các vector thành một mặt nhìn từ trên xuống.
4. **CNN 2D và head SSD.** Head kiểu ảnh vẽ cuboid: class, score, và bảy số.

Figure 2, Lang et al., PointPillars, CVPR 2019. KITTI test, Car, IoU 0,7, Moderate: AP_{3D} 74,99%.

CenterPoint: object là một tâm trên BEV

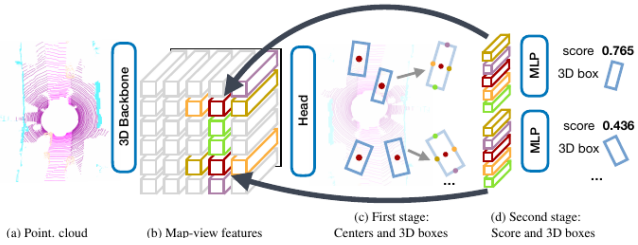


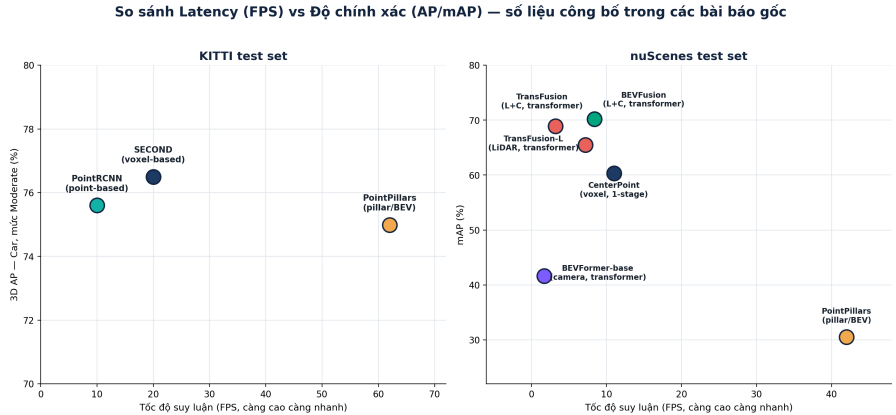
Figure 2, Yin et al., CenterPoint, CVPR 2021.

Cấu hình	Số đã báo
test, voxel	58,0 mAP / 65,5 NDS
val, pillar	50,3 mAP / 60,2 NDS
Waymo, 1 frame	69,0 mAPH L2

Hai dòng đầu là nuScenes; 11 FPS và 31 FPS trên Titan RTX, batch 1. Waymo là mAPH L2.

Encoder vẫn là pillar hoặc voxel. Head vẽ một heatmap tìm tâm, rồi hồi quy chiều cao, kích thước, yaw và vận tốc. Nhiều paper sau chỉ thay backbone, giữ head này. Tâm và vận tốc là cầu nối sang tracking — Ngày 14.

Latency và accuracy trên hình



KITTI và nuScenes là hai hệ đo. FPS lấy từ phần cứng từng bài. Đọc hướng: pillar nghiêng về tốc độ, voxel nghiêng về độ chính xác, trên đúng benchmark của từng bài.

Mọi họ đều ra class, score, và bảy số



Các họ chỉ khác cách biến điểm thành đặc trưng. Đầu ra mà người gắn nhãn cầm trên tay vẫn là class, score, và bảy số Ngày 12 đã học.

Score cao nói class khá chắc. Heading và đáy box là chuyện khác — phần tiếp theo nhắc đúng bảy số đó, rồi mới tới pipeline sinh ra chúng.

20 phút với ba họ vừa học

20 phút.

Chỉ ba họ:
Point-based
Voxel-based
PointPillars

1. Point-based giữ điểm thô. Hình học nào nó giữ được, và vì sao chậm trên cả phố?
2. VoxelNet cắt khối 3D. SECOND bỏ phần tính nào, và tốc độ đổi thế nào?
3. PointPillars gộp chiều cao thành pillar rồi đưa vào CNN 2D. Vì sao detector này nhanh hơn voxel 3D, và nó đánh đổi chiều cao cùng heading thế nào?
4. Cả ba họ trả về cái gì? Score cao bảo đảm phần nào, và heading thì sao?

Camera có dự đoán được 3D không?

04

Xe có thể chạy bằng camera. Ground truth của bài toán 3D đó vẫn đo từ LiDAR.

Tesla: trên xe là camera



Tesla Model 3. Carlquinn, Wikimedia Commons, CC BY-SA 4.0. Nóc không có LiDAR.

Tesla chạy nhận thức bằng camera. Xe khách không gắn LiDAR: nhiều camera, một stack vision, không có máy quét laser trên nóc.

Lưu ý: Ảnh này nói về cảm biến lúc xe đang chạy. Ground truth dùng để dạy và để chấm model 3D là 3D cuboid từ LiDAR.

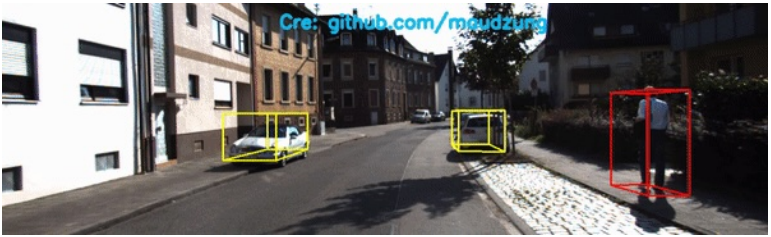
Cùng KITTI: LiDAR và camera

VINUNIVERSITY

Method	Car AP_{3D} Moderate	IoU	Nguồn
PointPillars (LiDAR)	**74,99%**	0,7	Lang et al., CVPR 2019
CaDDN (một camera)	13,41%	0,7	Reading et al., CVPR 2021

Cùng KITTI test, cùng class Car, cùng ngưỡng IoU. CaDDN là monocular — một ảnh duy nhất.

RTM3D: prediction từ một camera



Prediction của RTM3D trên ảnh KITTI. Ảnh: github.com/maudzung. Li et al., ECCV 2020.

1. Hộp vàng và đỏ là prediction của RTM3D, một camera-based 3D detector, chiếu lên camera.
2. LiDAR vẫn là ground truth để dạy và để chấm. Đối hộp camera với hộp LiDAR chỉ đúng khi calibration và synchronization đúng.
3. Vật xa hoặc bị che trên một ảnh thì dễ miss.
4. Heading dự đoán có thể lệch dù hộp đã gần đúng chỗ.

Ground truth của camera-based 3D detection là LiDAR

VINUNIVERSITY

Cuboid dùng để dạy và để chấm model camera là hộp 3D đo trên point cloud LiDAR, rồi người sửa lại. Model camera bị hỏi: hộp của bạn có khớp hộp LiDAR đó không.

Người gắn nhãn LiDAR đang viết đề thi. Xe Tesla cho thấy camera có thể lái. Các benchmark cho thấy khi cần một đáp án 3D bằng mét, ngành vẫn lấy LiDAR làm gốc.

Hai câu đứng cạnh nhau, không thay nhau: xe có thể chạy bằng camera; ground truth của bài toán camera-based 3D object detection là cuboid từ LiDAR.

Nhắc lại Ngày 12

05

Bảy số đã học. Lab 12 đã thực hành vẽ cuboid.

Bảy thông số

Tham số	Nghĩa	View ở lab 12
center (x, y, z)	Vị trí hộp, ego frame	Top: x, y · Side/Front: z
L, W, H	Dài, ngang, cao (mét)	Top: L, W · Side/Front: H
yaw	Hướng đầu, quanh trục z	Top
ground contact	Ràng buộc z, H, mặt đất	Side, Front

Mọi detector ở các slide trước trả về đúng bộ này, kèm class và score. Heading lệch 180° là hai nhãn khác nhau dù tâm và kích thước trùng. Object đứng yên vẫn có hướng đầu.

Kích thước điển hình: lab 12 đã dùng, hôm nay chỉ nhắc luật

VINUNIVERSITY

Dimension prior là kích thước một lớp hay gặp, khi điểm không phủ hết vật.

- Nhớ khoảng dài, ngang, cao thường gặp của lớp đó
- Ước phần bị che khi chỉ thấy một mặt điểm
- Nhận ra box quá to hoặc quá nhỏ so với lớp

- Không chọn class. Xe hay người do guideline chọn.
- Hộp ôm cả vật, kể cả chỗ bị che. Guideline VinFast chọn hai việc này.

Cụm điểm dài hơn kích thước điển hình thì tin điểm, sau khi đã loại hai vật dính làm một.

Khi box model trông sai: thứ tự review cũ



Heading sai kéo *L*/*W* sai. Đáy sai kéo *z* và *H* sai. Lab 12 đã đi thứ tự này trên vật dễ. Buổi lab sau dùng lại đúng thứ tự đó trên prediction — hôm nay chưa mở tool.

Quy trình pre-label bằng pre-trained model

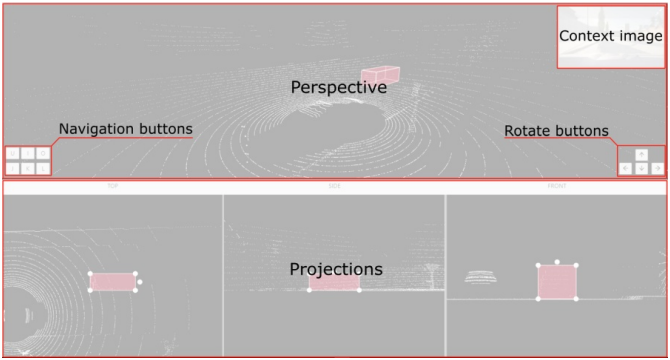
06

Model vẽ trước. Người sửa. Lỗi hệ toạ độ thì sửa cả batch, không kéo từng hộp.

Vì sao không vẽ từ một cloud trống

- Vẽ từng cuboid từ đầu chậm và đắt.
- PointPillars và các detector cùng họ đã định vị được xe, người, xe đạp ở mức dùng được trên dữ liệu gần KITTI/Waymo.
- Quy trình: chạy checkpoint có sẵn trên file `.pcd`, đưa box vào tool thành pre-annotation, annotator sửa.
- Điều kiện: cloud của mình và cloud lúc model được train phải cùng một quy ước trục *z*.

Quy trình nhìn thấy được



3D task workspace, docs.cvat.ai. CVAT, giấy phép MIT.

- Model vẽ cuboid trước. Người sửa hộp đó trên tool.
- Perspective là cả scene. Top, Side, Front là đúng thứ tự review đã học.
- Ô đỏ trên sơ đồ pipeline là buổi lab 4 giờ. Hôm nay chỉ học cách đọc hộp đó.

PointPillars: hệ quả với người sửa

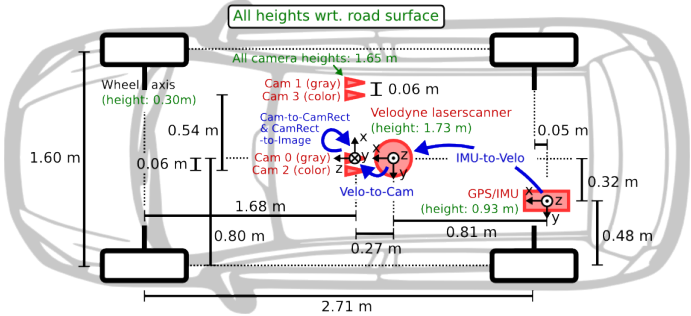
VINUNIVERSITY

Checkpoint pre-label của lab là PointPillars trên KITTI. Ba hệ quả sau là lý do annotator gặp họ này trước.

1. **Gộp chiều cao sớm.** Xe cao và xe thấp có thể chung một footprint trên ảnh giả. Chiều cao chi tiết không còn ở bước CNN.
2. **Cỡ pillar.** Lab đổi `voxel-size`. Ô $0{,}16$ m và ô $0{,}32$ m đổi số điểm trong một cột, rồi đổi hộp. Đó không phải cả batch bị cộng một hằng số theo z.
3. **Heading là chỗ yếu.** Score cao không kéo theo mũi tên đúng. So mũi tên với silhouette điểm, ở view Top.

Lưu ý: Output đã là cuboid bảy số. Người sửa cầm hộp đó, không cầm pillar.

Vì sao phải dịch *z*



Sensor setup, cvlibs.net/datasets/kitti. Geiger, Lenz, Stiller và Urtasun, IJRR 2013. Mọi chiều cao đo từ mặt đường.

Velodyne nằm trên nóc, cao $1{,}73$ m. Checkpoint KITTI coi mặt đất nằm dưới sensor khoảng đó.

Cloud của pipeline này để mặt đất ở $z \approx 0$, nên phải dịch *z* trước khi chạy model.

Hai gốc toạ độ, cùng một mặt đường

VINUNIVERSITY

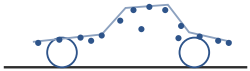
Gốc $(0, 0, 0)$ nằm ở mặt đất. Đỉnh histogram của z khoảng 0 đến $0{,}04$ m. Tâm hộp xe người đã vẽ nằm khoảng $0{,}5$–$1{,}7$ m.

Gốc ở Velodyne trên nóc, cao $1{,}73$ m so với đường (Geiger et al., IJRR 2013). Trong frame model, mặt đất ở $z \approx -1{,}73$.

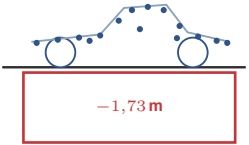
Gốc vehicle frame nằm trên mặt đất, giữa trục sau. Cloud và hộp checkpoint thường được train đã ở frame đó, nên mặt đất xấp xỉ $z = 0$. LiDAR nóc cao theo extrinsic từng xe. Không có hằng số $2{,}03$ m, và mặt đất không ở $z = -2{,}03$. Cửa sổ z khoảng -2 đến $+4$ m là khoảng đệm quanh gốc xe.

Đưa thẳng file `.pcd` gốc vào checkpoint KITTI là hỏi một model “mặt đất nằm dưới sensor $1{,}73$ m” bằng một cloud “mặt đất nằm ở 0”.

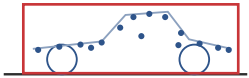
Ba kết cục của cùng một detector, cùng một scan



1. Không ra hộp
Model không trả
hộp dùng được.



2. Cả batch chìm
Quên đổi ngược:
hộp tụt dưới đường.



3. Hộp khớp điểm
Vào frame model,
ra frame nguồn.

Nhìn nghiêng, cùng một xe, cùng mặt đất. Xanh: điểm. Đường xám: mặt đất. Đỏ: hộp model. Slide sau là ba câu khi mở side view thật.

Đọc side view: ba câu

Slide trước là cùng một xe. Plot dày của cả con phố không cần đọc. Chỉ trả lời ba câu.

1. **Có hộp bám xe không?** Không có hộp trên cụm điểm: model không ra hộp dùng được. Đừng vẽ bù cho đủ số.
2. **Cả batch có cùng chìm một khoảng không?** Mọi hộp nằm dưới đường, lệch khoảng $1{,}73\,\text{m}$: quên đổi z ngược. Escalate cả batch. Kéo từng hộp lên là giấu lỗi.
3. **Hộp đã ngồi trên điểm chưa?** Đổi z hai chiều đúng thì hộp trùng cụm điểm. Lúc đó mới sửa heading và từng cạnh.

Lưu ý: Một hộp lệch là một xe. Mọi hộp lệch cùng một hằng số là pipeline.

Chỉ một phép tịnh tiến theo z

Trước khi chạy model

$$z_{\text{model}} = z_{\text{source}} - z_{\text{ground}} - h_{\text{sensor}}$$

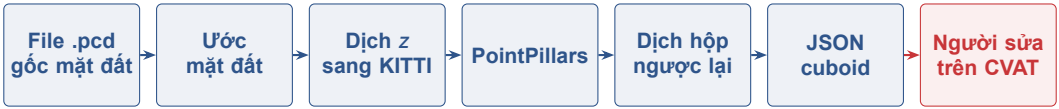
Sau khi có box, trước khi ghi ra tool

$$z_{\text{source}} = z_{\text{model}} + h_{\text{sensor}} + z_{\text{ground}}$$

- z_{ground} ước lượng từng scan, không giả sử đúng bằng 0
- h_{sensor} của checkpoint KITTI trong lab: $1{,}73$ m. Waymo không trừ $2{,}03$ m theo cùng công thức, vì gốc công bố đã nằm ở mặt đất
- x, y và yaw giữ nguyên trong pipeline minh họa này

Lưu ý: Điểm đi theo chiều thuận. Box đi theo chiều ngược. Quên một trong hai chiều là cả batch lệch cùng một hằng số.

Pipeline, từ file đến hộp chờ người sửa



Lab chạy PointPillars.

Ô đỏ là buổi lab 4 giờ. Hôm nay học để nhận ra khi nào phải dừng cả batch.

QC bằng mắt, trước khi đụng từng cạnh

1. **Cả batch nổi hoặc chìm cùng một khoảng.** Đó là lỗi transform. Escalate cả batch. Kéo 50 hộp về đất là giấu lỗi pipeline.
2. **Yaw.** Đầu hướng của pretrained detector là chỗ yếu nhất. So mũi tên với silhouette điểm, theo thứ tự Top đã học.
3. **False positive.** Cụm cây, cột, góc nhà có thể thành một "xe".
4. **Miss.** Class và tư thế hiếm trong KITTI/Waymo — vật cản, thú, xe bị che — model bỏ sót. Phải quét cả scan.
5. **Track ID** qua nhiều frame: để Ngày 14, khi có sequence.

Tổng kết — Key Takeaways

VINUNIVERSITY

Những ý chính cần nhớ khi đọc một cuboid do pretrained detector đề xuất

1. Sáu bài toán phổ biến trên LiDAR: classification, detection, segmentation, tracking, registration và SLAM. Hôm nay là 3D detection: cả scene, class và cuboid.

2. Thứ tự các họ: điểm, rồi voxel thưa, rồi pillar, rồi tâm trên BEV, rồi tinh lại bằng Transformer hoặc fusion. Đầu ra vẫn là class, score và 7 số cuboid.

3. Xe Tesla chạy bằng camera. Ground truth của camera-based 3D detection trên KITTI, nuScenes, Waymo là cuboid từ LiDAR.

4. Pre-label dịch *z* sang frame của checkpoint rồi dịch box ngược lại. Cả batch lệch cùng một hằng số là lỗi pipeline, escalate cả batch.

Tiếp theo & Bài tập

Lab: sửa prediction của pretrained detector

“Cả batch nổi cùng một khoảng — đó là lỗi pipeline, hay lỗi từng chiếc xe?”

- Nhớ bảy số và thứ tự Top, Side, Front khi một box model trông sai
- Ngày 14: một object giữ identity qua nhiều frame, và camera với LiDAR nhìn cùng một thế giới

Nguồn ảnh*

Mã	Nguồn
Ba dạng, điểm, pillar	Fig. 2, Lang et al., PointPillars, CVPR 2019
Voxel 3D	Fig. 2, Zhou và Tuzel, VoxelNet, CVPR 2018
Trends, latency	Sơ đồ tổng hợp của bài giảng
Classification	Fig. 1, Qi et al., PointNet, CVPR 2017
Segmentation	Behley et al., SemanticKITTI, semantic-kitti.org
Tracking	KITTI Tracking, Geiger et al., cvlibs.net
Registration	Open3D, tutorial ICP registration
Odometry	KITTI Odometry, Geiger et al., cvlibs.net
Giao lộ	Wikimedia, Ouster OS1-64, CC BY 4.0
Tesla Model 3	Carlquinn, Wikimedia, CC BY-SA 4.0
Ba kết cục	Sơ đồ bài giảng: xe nhìn nghiêng, ba vị trí hộp
Pipeline	Sơ đồ bài giảng, từ file .pcd đến CVAT
CVAT 3D	docs.cvat.ai, 3D task workspace, MIT
KITTI setup	cvlibs.net, Geiger et al., IJRR 2013
RTM3D	Ảnh prediction, github.com/maudzung

Kiến trúc	Figure
PointNet++	Fig. 2, Qi et al., NeurIPS 2017
VoxelNet	Fig. 2, Zhou và Tuzel, CVPR 2018
PointPillars	Fig. 2, Lang et al., CVPR 2019
CenterPoint	Fig. 2, Yin et al., CVPR 2021

Số accuracy và FPS trên từng slide lấy từ paper của chính model đó.

Tài liệu tham khảo

1. **VinFast — Official Robotaxi 3D/LiDAR guideline** (nguồn rule duy nhất).
2. **Lang et al. (CVPR 2019)** — PointPillars. **Yin et al. (CVPR 2021)** — CenterPoint.
3. **Reading et al. (CVPR 2021)** — CaDDN, monocular 3D detection. Số AP_{3D} camera-only trên KITTI. **Li et al. (ECCV 2020)** — RTM3D, prediction 3D từ một camera.
4. **Qi et al.** — PointNet, CVPR 2017; PointNet++, NeurIPS 2017. **Yan et al.** — SECOND, Sensors 2018.
5. **Geiger, Lenz, Stiller và Urtasun (IJRR 2013)** — KITTI. Velodyne cao $1{,}73\,\mathrm{m}$. **Waymo Open Dataset** — vehicle frame: gốc trên mặt đất, giữa trục sau.

VINUNIVERSITY

Cảm ơn!

Buổi lab sẽ mở prediction và sửa cuboid. Hôm nay mang theo hai câu:

đầu ra của mọi họ model là bảy số đã học,
và một batch lệch cùng một hằng số là lỗi hệ toạ độ.